GDA Command Platform Rescue Map

Date: April 27, 2026

Executive Decision

React is the primary and only forward product UI for the GDA Command Platform.

Retool is no longer part of the product path. Because the Retool build began only as a short-term replacement attempt for React, it should be treated as an archive/reference only, not an admin product to maintain.

n8n remains the automation engine. Postgres remains the system of record. The next architecture improvement is a small API gateway between React and n8n/Postgres.

Goal

Make GDA Command reliable enough for daily business-development, capture, proposal, and executive operating use without requiring Shawn to babysit workflows.

The near-term objective is not a full rebuild. The near-term objective is controlled stabilization.

Current System

Primary UI

Component	Decision	Notes
React GDA Command Center	Keep	Main product UI going forward.
Retool GDA Command Platform	Archive / stop work	Do not mirror React in Retool. Keep only as a temporary reference until React is stable.

Backend and Automation

Component	Decision	Notes
n8n	Keep	Automation, scheduled jobs, enrichment, alerts, AI workflows, integrations.
Postgres	Keep	System of record for opportunities, intelligence, action history, risks, caches, and analytics.
API Gateway	Add	Small backend layer needed to stabilize React and reduce direct dependency on n8n webhook behavior.
QA Agent	Keep and improve	Should become the control tower for inventory, smoke tests, failure detection, and approved fixes.

Verified Baseline

As of the latest stabilized session:

Area	Status
React-used workflow smoke group	10/10 passing
Retool-used workflow smoke group	4/4 passing
n8n smoke-test internal checks	5/5 passing
`GDA.cron.system-watchdog`	Green
`GDA.cron.pipeline-health-digest`	Green
`GDA.cron.pipeline-coverage-check`	Green
`GDA.cron.capture-gate-review`	Green
`GDA.sched.golden-dome-monitor`	Fixed and passing
`GDA.sched.dept-market-refresh`	Fixed and passing
`GDA.cron.competitor-crawler`	Fixed and manually tested green
`GDA.cron.comp-intel-daily-growth`	Manually tested green
`GDA.sched.dept-opp-sweep`	Fixed and manually tested green
`GDA.cron.stage-auto-promote`	Tested safely; no promotable rows
Controlled n8n fix agent	Working for inventory, inspection, proposal, and approved patch apply
React source stabilization patch	Deployed to live React; build passed; live smoke verified zero direct n8n webhook calls on tested routes
React sidebar route smoke pass	23/23 sidebar pages clicked through; zero direct n8n webhook calls; zero browser console errors
React smoke-test automation kit	Created and verified; `gda-react-smoke-tests.zip` runs repeatable 23-route Playwright smoke checks
React functional smoke checks	8/8 high-value non-destructive checks passing; zero direct n8n webhook calls; zero console errors
React API proxy smoke checks	10/10 React-critical proxy endpoints passing; zero auth failures; zero empty responses
React full smoke bundle	`npm run all` passed: sidebar 23/23, functional 8/8, API proxy 10/10
React daily baseline command	`npm run baseline` created and passing; writes one combined executive report

Area	Status
GDA ops daily baseline command	`npm run ops` created; React checks pass and optional n8n QA checks run when local QA agent URL/key are set

The temporary open QA operator endpoint later returned 404, likely because it was disabled or protected. That is the correct security posture. Use the secured QA agent long-term.

React Source Stabilization Patch - April 27, 2026

The uploaded React source ZIP was patched to remove direct frontend-to-n8n webhook calls and route them through relative `/api/<webhook-path>` proxy paths. The patch removed direct calls to `gda-capture-plan`, `gda-ai-chat`, `gda-deep-research`, `gda-deep-research-history`, `gda-capture-suite`, `gda-morning-briefing`, and `gda-semantic-search` from active source files. Frontend webhook auth headers such as `x-gda-key`, `x-gda-auth`, and `X-GDA-Key` were removed from those calls.

Patched artifact: `gda-command-react-stabilized.zip`. Report artifact: `gda-react-stabilization-report.md`. Validation: `npm ci` succeeded and `npm run build` succeeded.

Deployment note: the production deploy endpoint returned timeout/502-style responses during the build step, but the active source and production bundle were updated successfully. The deployed bundle was verified live after build.

Live smoke result: Launchpad, Deep Research, Proposal Factory, Financial Bible, and Morning Brief loaded successfully. Across those tested routes there were zero direct browser calls to `n8n.csr-llc.tech/webhook`, zero browser console errors, and zero failed API requests. The routes now call the server-side proxy under `https://gda.csr-llc.tech/api/...`.

Remaining action: continue expanding React route coverage and remove stale old JS asset files from the production asset folder so future grep checks are cleaner.

React Sidebar Route Coverage - April 27, 2026

Live sidebar smoke testing clicked through 23 React sidebar pages: Launchpad, AI Agent Chat, Daily SITREP, Morning Brief, Intel Feed, Deep Research, Meeting Notes, Competitive Intel, Bid Strategy, OODA Loop, Opportunity Tracker, Pwin Calculator, Capture Plan, Compliance Matrix, Document Hub & PPT, Email Drafter, Proposal Review, Proposal Factory, Risk Register, Financial Bible, Platform Health, Platform Guide, and Prompt Architect.

Result: all 23 routes loaded without browser console errors and without direct browser calls to `n8n.csr-llc.tech/webhook`. API traffic stayed under the server-side proxy path `https://gda.csr-llc.tech/api/...`.

Observed note: one `gda-action-history` request showed `net::ERR_ABORTED` during rapid page-to-page navigation. That appears to be a navigation abort rather than a server-side failure. It should be watched, but it is not the same class of issue as the prior direct n8n webhook/auth failures.

Next React stabilization step: turn this sidebar route pass into a repeatable Playwright test suite and add deeper functional checks for forms/buttons on the highest-value pages.

React Smoke-Test Automation Kit - April 27, 2026

A standalone read-only Playwright kit was created as `gda-react-smoke-tests.zip`. It can be run from PowerShell after React deploys to test the live app without requiring n8n API keys or webhook keys.

The kit checks 23 sidebar routes and fails if it sees direct browser calls to `n8n.csr-llc.tech/webhook`, browser console errors, blocking failed API requests, or unclickable sidebar routes.

Initial sandbox run result: PASS. Routes tested: 23. Routes passed: 23. Direct n8n webhook calls: 0. Console errors: 0. Blocking failed requests: 0.

Recommended operating habit: run `npm run smoke` from the smoke-test folder after every React deploy and before relying on the app for daily work.

The kit was expanded with `npm run functional`, a non-destructive set of high-value page checks for Launchpad, Opportunity Tracker, Platform Health, Deep Research, Proposal Factory, Financial Bible, Risk Register, and Morning Brief. Initial result: PASS. Checks tested: 8. Checks passed: 8. Direct n8n webhook calls: 0. Console errors: 0. Blocking failed requests: 0.

The kit was expanded again with `npm run api`, a read-only API proxy check that calls React-critical server-side proxy routes without exposing n8n webhook keys in the browser. Initial result: PASS. Endpoints tested: 10. Endpoints passed: 10. Auth failures: 0. Empty responses: 0. HTML responses: 0.

API proxy endpoints verified: `gda-dashboard-mega`, `gda-trends`, `gda-daily-actions`, `gda-opp-tracker`, `gda-launchpad-funnel`, `gda-deep-research-history`, `gda-capture-plan`, `gda-risk`, `gda-platform-health`, and `gda-morning-briefing`.

Use `npm run all` to run the 23-route sidebar pass, the functional smoke checks, and the API proxy checks together. Latest full bundle result: PASS. Sidebar routes: 23/23. Functional checks: 8/8. API proxy endpoints: 10/10.

The kit now includes `npm run baseline`, the recommended daily confidence command. It runs the sidebar, functional, and API proxy checks and writes one combined report: `reports/gda-react-daily-baseline-latest.md`. Latest daily baseline result: PASS. Sidebar routes: 23/23. Functional checks: 8/8. API endpoints: 10/10. Direct n8n webhook calls: 0. Browser console errors: 0. Blocking failed requests: 0. API auth failures: 0.

GDA Ops Daily Baseline - April 27, 2026

The smoke-test kit now includes `npm run ops`, a read-only operations baseline command. It runs the React daily baseline first. If local PowerShell environment variables `GDA_QA_AGENT_WEBHOOK_URL` and `GDA_QA_AGENT_KEY` are set, it also checks secured QA agent

health, React-used n8n workflows, Retool-used n8n workflows, and latest failed n8n executions.

If those local secrets are not set, the n8n QA checks are marked `SKIPPED` rather than `FAIL`, so React confidence checks remain usable without exposing secrets in the kit. The helper `run-ops-baseline-windows.ps1` prompts for the QA agent URL/key when needed.

The kit also includes `setup-qa-agent-windows.ps1`, a one-time Windows helper that saves `GDA_QA_AGENT_WEBHOOK_URL` and `GDA_QA_AGENT_KEY` to the Windows user environment and tests QA-agent health. After that setup, the normal daily command is `npm run ops`.

Latest sandbox validation without local QA secrets: NEEDS_REVIEW because n8n QA checks were skipped, with 0 hard failures. React checks passed: sidebar 23/23, functional 8/8, API proxy 10/10.

Known Architecture Problem

The platform works, but too much responsibility is concentrated inside n8n.

Current pattern:

```
React / Retool
  -> nginx proxy / direct resources
  -> n8n webhooks
  -> scattered code-node auth
  -> Postgres / external APIs / Telegram
```

This caused:

- stale hardcoded keys
- duplicate auth checks
- workflows returning HTTP 200 while failing internally
- brittle Postgres query styles
- unclear side effects
- no consistent dry-run mode
- hard-to-debug React/Retool inconsistencies

Target pattern:

```
React
  -> GDA API Gateway
  -> Postgres for core reads/writes
  -> n8n for automation and long-running jobs
  -> QA agent for monitoring and controlled execution
```

Retool should not be maintained as a parallel UI. If anything useful exists in Retool, copy the idea into React and then leave Retool archived:

```
Retool
  -> archive/reference only
  -> no new product work
```

UI Strategy

Keep React

React should own:

- Launchpad / Command Center
- Opportunity Tracker
- Platform Health
- Action Items
- Risk Register
- Daily SITREP / Morning Brief
- AI Agent Chat
- Capture and proposal tools
- Executive dashboards
- Partner/customer-facing views if needed

Stop Retool Work

- Do not mirror React in Retool.
- Do not build new Retool screens.
- Do not maintain Retool as a second product.
- Keep Retool only as a temporary archive/reference.
- Copy any useful Retool ideas into React, then ignore Retool.

Stop Doing

- Do not build new user-facing product features in Retool.
- Do not let React and Retool diverge as two separate products.
- Do not expose business users to both as equal options.
- Do not spend time keeping Retool in sync with React.

Workflow Risk Model

Every n8n workflow should be categorized before being run by an agent.

Category	Meaning	Auto-run?
Read-only	Reads DB or returns status only	Yes
Writes cache/report	Writes non-critical summary/cache rows	Usually yes
Writes business data	Changes opportunities, capture plans, risks, actions	Approval required
External API-heavy	Calls Perplexity, Anthropic, SAM.gov, GovTribe, etc.	Approval required
Sends message	Telegram/email/Slack/customer-facing send	Approval required
Dangerous	Deletes, bulk updates, changes stages, deploys code	Human approval required every time

Workflows Already Touched

Fixed or Stabilized

Workflow	Status / Fix
`GDA.api.dashboard-mega`	Removed stale internal auth guard.
`GDA.api.platform-health`	Removed stale internal auth guard.
`GDA.api.intel-feed`	Removed stale internal auth guard.
`GDA.util.smoke-test`	Updated stale webhook key; now 5/5 internal checks pass.
`GDA.error.handler`	Corrected logging query.
`GDA.intel.morning-briefing-v1`	Fixed stale key in data-learn call.
`GDA.api.data-learn`	Removed stale internal auth guard.
`GDA.cron.weekly-comp-scan`	Fixed unsafe JSON SQL interpolation.
`GDA.cron.nightly-fy-revenue-calc`	Fixed `gda_mega_cache` column/type assumptions.
`GDA.cron.data-retention`	Fixed timestamp/created_at column assumptions.
`GDA.api.ndaa-far-ingest`	Removed stale internal auth guard.
`GDA.api.morning-briefing`	Removed stale internal auth check.
`GDA.sched.golden-dome-monitor`	Removed stale internal auth guard.
`GDA.sched.dept-market-refresh`	Removed stale internal auth guard; retested passing.

Workflow	Status / Fix
`GDA.cron.competitor-crawler`	Parameterized `Load Last Crawl` and `Store Crawl`; enabled `Always Output Data` on `Load Last Crawl`; manually tested green through final node.
`GDA.cron.comp-intel-daily-growth`	Manually tested green through final node.
`GDA.sched.dept-opp-sweep`	Removed stale internal auth guard; manually tested green.
`GDA.cron.stage-auto-promote`	Parameterized promote replacement; manually tested safe stop with no promotable rows.
`GDA.auto.e2e-gemini-report`	Removed stale hardcoded key usage; switched e2e calls and Telegram notify to the server-side `/api` proxy path.

Held For Controlled Review

Workflow	Reason
`GDA.cron.competitor-crawler`	Still external API-heavy and can send Telegram; query issues fixed and manual path tested green. Add dry-run/preview before routine unattended runs.
`GDA.cron.comp-intel-daily-growth`	External API-heavy; manual path tested green. Still needs stronger preview/deduping before unattended expansion.
`GDA.sched.dept-opp-sweep`	Writes opportunity data; auth issue fixed and manual path tested green. Still needs report-only mode before routine unattended runs.
`GDA.cron.stage-auto-promote`	Changes opportunity stages. Safe test showed no promotable rows. Must always preview exact candidates before running.

Required Platform Standards

Auth Standard

There should be one external auth layer per workflow endpoint.

Preferred:

- Use n8n webhook credential-level auth for webhook protection.
- Remove hardcoded code-node checks like:

```
if (key !== 'gda-api-2026-secure') throw new Error('Unauthorized');
```

Exception:

- Keep explicit auth only for especially dangerous internal admin/deploy workflows, and store expected secrets in environment variables, not hardcoded code.

Response Standard

All API-style workflows should return:

```
{
  "ok": true,
  "workflow": "GDA.api.example",
  "status": "success",
  "data": {},
  "errors": []
}
```

Failures should return:

```
{
  "ok": false,
  "workflow": "GDA.api.example",
  "status": "error",
  "error": "Human readable error",
  "details": {}
}
```

Dry-Run Standard

Any workflow that writes business data must support:

```
{
  "dryRun": true
}
```

or:

```
{
  "mode": "preview"
}
```

Dry-run mode should return what would change without changing it.

Required for:

- `GDA.cron.stage-auto-promote`
- `GDA.sched.dept-opp-sweep`
- `GDA.cron.comp-intel-daily-growth`
- any workflow that deletes, inserts, upserts, promotes, sends, or deploys

Postgres Query Standard

Avoid mixing query styles.

Known problem:

```
VALUES ($1, $2, $3)
```

Some current n8n Postgres nodes are not configured to pass query parameters this way.

Short-term standard:

- Use n8n expressions carefully.
- Escape single quotes.
- Cast JSON/text explicitly.

Long-term standard:

- Move core database writes behind the GDA API gateway where parameterized queries are normal and testable.

Target Architecture

Near-Term

```
React
  -> nginx server-side proxy
  -> n8n webhooks / Postgres-backed APIs

Retool
  -> archive/reference only

n8n
  -> automations, scheduled jobs, enrichment, alerts

QA Agent
  -> inventory, smoke tests, latest failures
```

Target

```
React
  -> GDA API Gateway
      -> Postgres for normal app reads/writes
      -> n8n for background jobs and automations
      -> external APIs through controlled services

Retool
  -> archived; not a maintained app
```

```
QA Agent
  -> monitors React, API gateway, n8n, and Postgres health
```

API Gateway Scope

Build a small backend service, not a giant rewrite.

Initial endpoints:

Endpoint	Purpose
`/api/health`	Platform health and dependency status
`/api/dashboard`	Launchpad dashboard data
`/api/opportunities`	Opportunity tracker read/search
`/api/funnel`	Pipeline funnel data
`/api/trends`	Trend data
`/api/actions`	Action items
`/api/risks`	Risk register
`/api/qa/status`	QA agent summary

The API gateway should own:

- auth
- input validation
- response format
- error handling
- rate limiting
- stable React-facing routes

n8n should own:

- scheduled jobs
- long-running AI tasks
- external data ingestion
- notifications
- enrichment
- workflow orchestration

Now / Next / Later Roadmap

Now

- Treat React as the main UI.
- Stop Retool work and treat it as archive/reference only.
- Keep QA agent secured.
- Finish workflow risk classification.
- Fix held workflow query/auth issues before running.
- Add dry-run mode to dangerous workflows.
- Keep React, Retool, and smoke tests green.

Next

- Build the GDA API gateway.
- Move React critical endpoints behind the API gateway.
- Create a workflow registry.
- Add daily monitoring.
- Add standard response shapes to API-style workflows.
- Add dry-run support to all workflows that write/send/change state.

Later

- Pull any useful Retool-only ideas into React.
- Retire Retool from daily work.
- Move core business writes out of n8n and into API gateway services.
- Add richer role-based permissions.
- Add audit trails for all human-approved workflow runs.

Workflow Registry Template

Each workflow should eventually have a registry entry:

```
name:
id:
owner:
purpose:
trigger_type:
inputs:
outputs:
reads_tables:
writes_tables:
external_apis:
sends_messages:
```

```
changes_business_state:
safe_to_auto_test:
requires_human_approval:
dry_run_supported:
last_known_status:
notes:
```

Daily Operating Checklist

Run or review:

1. React daily baseline: `npm run baseline` from the `gda-react-smoke-tests` folder.
2. GDA ops baseline: `npm run ops` from the `gda-react-smoke-tests` folder after setting `GDA_QA_AGENT_WEBHOOK_URL` and `GDA_QA_AGENT_KEY` locally.
3. Critical n8n workflow group.
4. Platform health endpoint.
5. Smoke-test internal checks.

Do not run without approval:

- workflows that send Telegram/email
- workflows that call paid external APIs
- workflows that upsert opportunities
- workflows that change opportunity stages
- deploy/fix workflows
- delete/cleanup workflows beyond known safe retention jobs

Immediate Next Session

Recommended next prompt:

```
Continue with GDA API gateway stabilization
```

First task:

- Run daily baseline checks:
  - React-used workflow smoke group
  - Retool-used workflow smoke group
  - latest failed executions
  - platform health
  - smoke-test internal checks

Then:

- Add dry-run/preview support to workflows that write business data or send notifications.

- Start moving React-critical routes behind a small GDA API gateway.

Bottom Line

The project should not be scrapped.

The product direction is:

```
React = product
Retool = archive/reference only
n8n = automation engine
Postgres = system of record
API gateway = stability layer
QA agent = control tower
```

The work ahead is stabilization and consolidation, not starting over.